Engineering Design Report

Author: Abnik Ahilasamy

Date: May 2026

Repository: `github.com/kinba09/aker_RAG`

Contents

1. Problem Statement and System Constraints

Many enterprise systems contain two fundamentally different forms of information. Structured operational data lives inside relational databases, while unstructured knowledge exists across documentation, websites, reports, and internal content repositories. Building a reliable AI assistant requires reasoning across both worlds while maintaining accuracy, traceability, and low latency. On one side: structured, transactional rent-roll data — units, charges, balances, lease dates — exported from property management software as Excel files. On the other: unstructured, qualitative information published on each property's public website — amenities, floor plans, neighbourhood guides, community policies. A leasing agent or asset manager asking "what is the average balance for May 2025" and "what amenities does this property offer" is querying both worlds in a single workflow.

The challenge is to build a single question-answering interface that serves both types of query, scoped strictly to a single active property per session, reliably enough to be trusted in daily operations.

1.1 Hard Constraints at the Outset

The system was designed under three non-negotiable constraints that shaped every architectural decision that followed:

1. **Zero cost infrastructure.** All LLM inference, embeddings, and compute had to be free-tier or self-hosted. No API spend.

2. **Production-grade reliability.** Outputs for structured queries must be deterministic — the same question asked twice must return the same answer.

3. **Strict data tenancy.** Every query, every retrieval, every SQL execution must be scoped to one property code. Cross-property data leakage is a critical failure.

These three constraints are in tension with each other in interesting ways, and every design decision in this report can ultimately be traced back to resolving that tension.

2. The Pivotal Design Decision: Determinism Over Intelligence

When the initial architecture was being sketched, the obvious approach was to build a fully agentic system: give an LLM the database schema and let it write SQL on the fly for every query. This is how most RAG-plus-SQL tutorials are written, and frameworks like LangChain make it easy to wire up. The system would have felt elegant — one model handling everything.

The problem is that an LLM writing SQL is a non-deterministic system. Ask it the same question twice and you may get two different SQL queries that, by accident or by semantic drift, return different results. In a property management context, where a leasing agent

needs to trust that "highest balances" today means the same thing it meant yesterday, this is not an acceptable failure mode.

Design Decision

Context: LLM-generated SQL would be flexible and developer-friendly.

Problem: Non-deterministic output is fundamentally incompatible with financial data retrieval, where auditability and repeatability are required.

Decision: Build a library of deterministic SQL templates covering the common intent space. LLM-generated SQL becomes a *governed fallback* for queries that fall outside the template library, not the primary path.

Outcome: The system now has two SQL modes: `template` (deterministic, zero-latency, auditable) and `llm_generated_validated` (flexible, guardrailed, with provenance tracking). Every SQL response records which mode was used.

This decision also has a direct impact on the information retrieval framing. The system deliberately optimises for **precision over recall**. For financial rent-roll data, the dataset is bounded and the user needs the exact right rows — not a fuzzy approximation. High recall at the cost of precision would mean returning plausible-but-wrong data, which is worse than returning nothing.

3. Data Architecture

3.1 The Dual-Database Architecture

Two fundamentally different data types require two different storage strategies. Rather than forcing both into a single store (a common over-engineering trap), the system separates them cleanly.

3.1.1 Structured Data: MySQL

The rent-roll data, originally delivered as Excel files, contains: properties, rent roll snapshots (one per property per month), individual unit records, and per-unit charge breakdowns. The raw Excel files were significantly messy — single rows contained multiple fields, headers overlapped across multiple rows, and the field naming was inconsistent. Cleaning this data to a normalised relational schema was itself a significant engineering task.

Table	Purpose
`properties`	Master property registry, keyed by `property_code`
`rent_roll_snapshots`	Monthly snapshots; one row per (property, month)
`rent_roll_units`	Unit-level records: sq ft, move-in, lease dates, market rent, balance
`rent_roll_unit_charges`	Itemised charges per unit: security deposit, parking, utilities, etc.
`property_web_sources`	Website URL mapping per property code

3.1.2 Unstructured Data: Qdrant Vector Database

Domain-specific documentation and knowledge sources — scraped, cleaned, and chunked — lives in a separate Qdrant collection. The decision to keep this separate from MySQL was deliberate. Adding website content as new columns or rows in the relational schema would create massive redundancy (one website describing a property would duplicate across every unit record) and pollute the precision-optimised relational model with unstructured text.

A fresh collection with a simple, clean schema solved both problems.

3.2 Why the Data Split Was Not Optional

The temptation was to add website content directly into MySQL with extra fields. Three problems ruled this out:

1. **Redundancy at scale.** A property with 200 units would store the same website content 200 times.
2. **Schema pollution.** Mixing a 900-character text chunk with numeric financial fields breaks the relational model's assumptions and defeats the template SQL approach.
3. **Retrieval mismatch.** Relational databases use index-based exact-match or range queries. Website content needs semantic similarity search — a fundamentally different access pattern.

4. Vector Database Selection: Why Qdrant

With the decision to use a vector store made, five candidates were evaluated. The evaluation was not academic — each had to be assessed against the real constraints: free/self-hosted, strong metadata filtering, production latency, and operational simplicity in a Docker Compose environment.

Table 1: Vector Database Candidate Evaluation

Database	Self-Hosted	Metadata Filter	SQL Integration	Managed Cost	Ops Complexity
Qdrant	✓	✓ (strong)	×	Free	Low
pgvector	✓	✓	✓ (native)	Free	Very Low
Weaviate	✓	✓	×	Free	High
Pinecone	×	✓	×	Paid	None
Milvus	✓	✓	×	Free	Very High

pgvector was the most compelling alternative. Its native PostgreSQL integration would have allowed SQL and vector queries in the same connection. However, this system is not using PostgreSQL for structured data — it uses MySQL. The integration advantage disappears. Furthermore, pgvector's approximate nearest-neighbour (ANN) ceiling is lower than Qdrant's at scale, and the query pattern here is "chunk-level semantic retrieval with hard property scope," which is exactly Qdrant's design target.

Pinecone was ruled out immediately by the zero-cost constraint and vendor lock-in.

Weaviate has rich hybrid search capabilities but its operational complexity — schema definition, module configuration — was disproportionate to the use case.

Milvus offers the strongest throughput at scale but requires the heaviest operational footprint, which is unjustifiable for a system of this size.

Design Decision

Decision: Qdrant, for its combination of strong metadata filtering, low operational overhead, Docker-native deployment, and excellent latency on sub-million-scale vector collections. Its `property_code` metadata filter provides the hard tenancy isolation the system requires.

5. Web Crawling and Data Cleaning

5.1 The Crawling Problem

Raw HTML from property websites is almost entirely noise. CSS class names, navigation menus, cookie banners, JavaScript payloads, tracking scripts — the useful text typically constitutes less than 20% of the raw page content. If this content is embedded and stored without cleaning, the vector index is effectively poisoned: retrievals return semantically irrelevant chunks that happen to contain common HTML tokens.

Every property website was crawled (not just sampled), and the crawl output went through aggressive cleaning:

- CSS `<style>` blocks and all inline styles stripped
- `<script>` tags and JavaScript removed
- Navigation and footer boilerplate excluded
- Whitespace normalised and encoding cleaned

The result was clean prose text ready for chunking and embedding.

5.2 Chunking Strategy

With clean text in hand, the chunking approach becomes a meaningful engineering choice. Three strategies were considered:

Table 2: Chunking Strategy Comparison

Strategy	Advantages	Disadvantages
Fixed-size character	Simple, fast, deterministic, controllable	Can split mid-sentence; no semantic awareness
Token-aware	Respects LLM context limits exactly	Requires tokeniser; more complex pipeline
Structure-aware	Respects headings/paragraphs; best retrieval quality	Complex parsing; fragile on inconsistent HTML

Chosen: Fixed-size character chunking with `chunk_size=900` and `overlap=150`, implemented in `crawl_property_sites.py`.

The overlap of 150 characters ensures that a sentence split across a chunk boundary is represented in both chunks, preventing context loss at retrieval time.

Design Decision

Decision: Fixed-size chunking for the first ingestion pass. It is deterministic, operationally simple, and good enough for the retrieval quality needed. Structure-aware chunking (splitting on heading/paragraph boundaries with token-awareness) is the natural next upgrade and is already identified as the improvement path.

5.3 Chunk Metadata and the Graph RAG Linkage

Each stored chunk carries the following metadata fields:

```
{
    "property_code": "115R",
    "property_name": "...",
    "source_url": "https://...",
    "chunk_id": "115R_chunk_0042",
    "text": "...",
    "crawled_at": "2025-05-01T14:32:00Z"
}
```

The `property_code` field is the primary tenancy key — it is the metadata filter applied on every Qdrant query. But the combination of `source_url` and `chunk_id` also provides forward compatibility: if the system were extended with a graph layer, each chunk can be linked back to its source document and to the structured property record in MySQL by `property_code`. This technique was drawn from graph RAG implementations where chunked unstructured data needs bidirectional linkage to structured records.

6. Embedding Layer: From API to Local

The embedding layer went through two iterations, each driven by a real failure.

Iteration 1: Google Generative AI Embeddings. The initial design used Google's free

embedding API (`GoogleGenerativeAIEmbeddings`). This worked briefly before hitting quota limits and a Google-side model availability issue that was difficult to diagnose. The failure was opaque — the API was returning errors that did not clearly indicate quota exhaustion, costing significant debugging time.

Iteration 2: Local Ollama Embeddings. The solution was to switch to a local embedding model running inside Ollama: `nomic-embed-text-v2-moe`. This model runs on the host machine and is accessible from Docker containers via `host.docker.internal:11434`. Zero API cost, zero quota limits, consistent availability.

The system retains both providers in its configuration and can be switched via the `EMBEDDING_PROVIDER` environment variable:

```
EMBEDDING_PROVIDER=ollama
OLLAMA_EMBED_MODEL=nomic-embed-text-v2-moe
# or
EMBEDDING_PROVIDER=google
GOOGLE_EMBEDDING_MODEL=models/gemini-embedding-001
```

Engineering Lesson

Free-tier cloud APIs are attractive on paper but unreliable as infrastructure dependencies. Where a self-hosted alternative exists at comparable quality (as `nomic-embed-text-v2-moe` does for general-purpose embeddings), self-hosting eliminates an entire class of non-deterministic failure.

7. Agent Orchestration: LangGraph and the Routing Architecture

7.1 Framework Selection: Why LangGraph over CrewAI

Several orchestration frameworks were evaluated: LangGraph, CrewAI, and direct SDK usage (Anthropic/OpenAI clients).

CrewAI was attractive for its simplicity — defining agents and tasks reads almost like natural language. However, CrewAI abstracts away the execution graph, making it difficult to enforce the strict routing logic this system requires. When a production system needs to guarantee that a financial query *always* goes through SQL validation before touching the LLM, CrewAI's opaque scheduling becomes a liability.

Direct SDK usage (e.g., calling the Anthropic API with tool use) was considered but creates vendor lock-in. A system built on Claude's tool-calling API cannot easily swap to Gemini or an open model, which is a problem when the constraint is zero-cost operation.

LangGraph exposes the execution graph explicitly. Nodes are Python functions; edges are typed transitions. This means routing logic, state management, and fallback behaviour can all be written, tested, and debugged as ordinary code.

7.2 The Four-Node Graph

The LangGraph orchestrator implements a four-node directed graph:

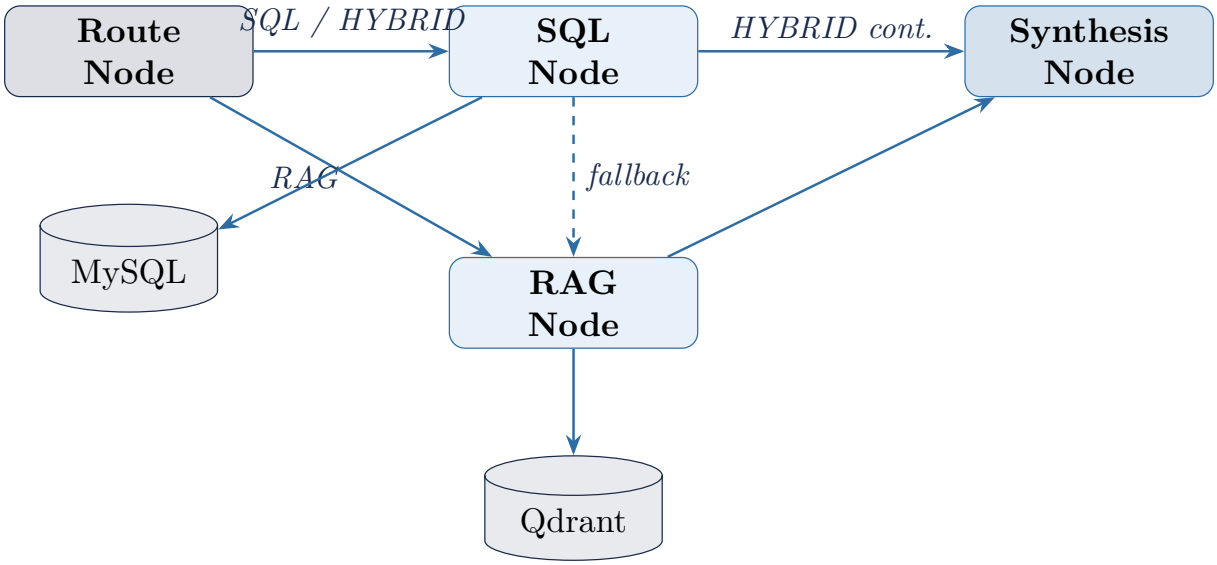

Figure 1: LangGraph orchestration graph. The dashed edge represents the SQL-first hybrid continuation to RAG.

Route Node: Classifies the incoming query as `SQL`, `RAG`, or `HYBRID` based on intent detection. Financial/operational queries (balances, occupancy, lease dates) route to SQL. Website/amenity queries route to RAG. Queries spanning both ("KPI summary and website highlights") route as HYBRID.

SQL Node: Executes deterministic SQL templates or the governed LLM-SQL fallback. Returns structured data with full provenance metadata.

RAG Node: Embeds the user query, queries Qdrant with a hard `property_code` filter, and returns the top-k semantically matching chunks with citations.

Synthesis Node: Receives data from one or both upstream nodes and produces the final `answer_markdown`, `ui_blocks`, and `citations` response structure.

7.3 The Critical Architecture Pivot: SQL-First, Not RAG-First

The initial design routed to RAG and SQL in parallel, then synthesised. A subtle but important change made to this: for HYBRID queries, **SQL executes first**, and its structured output is passed into the RAG context window before vector retrieval happens.

The reasons are both quality and latency:

1. **Grounding.** The synthesis LLM has access to the precise, verified structured data before it reads unstructured website chunks. This prevents the LLM from hallucinating a number that was already retrievable exactly.

2. **Latency UX.** Deterministic SQL executes in under 100ms. By running SQL first and streaming the result, the system can return the first token to the user almost immediately. The slower RAG retrieval and LLM synthesis happen in the background.

Insight on Latency Psychology: There are two distinct latency metrics that matter for conversational AI: Time to First Token (TTFT) and Token Throughput (TPS). Human psychology responds differently to each. A slightly slower token stream is tolerable — the user can read it in real time. But a long wait for the *first* token creates anxiety and perceived slowness. The SQL-first architecture minimises TTFT by returning verified structured data before the LLM generation begins. The synthesis prompt was also deliberately kept concise to further reduce TTFT.

8. The SQL Layer: Deterministic Templates and Governed Fallback

8.1 Deterministic Intent Coverage

The SQL node implements a template library covering the most common property management intents:

Intent	Description
`kpi_summary`	Aggregate KPIs for the active property/month
`occupancy`	Occupancy rate and vacant unit count
`vacant_units`	List of currently vacant units
`highest_balances`	Residents ranked by outstanding balance
`unit_detail`	All fields for a specific unit number
`unit_scalar`	Single field lookup (sq ft, market rent, etc.)
`rent_by_unit`	Rent charge breakdown per unit
`deposits`	Security deposit summary
`lease_charges`	Itemised charge breakdown
`leases_expiring`	Leases expiring in a given month
`leases_expiring_next`	Leases expiring next calendar month

8.2 Month Resolution Logic

A non-trivial amount of the SQL layer's complexity is in date resolution. Users do not say "2025-05" — they say "May," "last month," or nothing at all. The system implements layered month resolution:

1. Default to the **latest available snapshot month** for the active property.
2. Recognise explicit **ISO format** (`2025-05`).
3. Parse **month names** (`May 2025`, `May`).
4. Apply **all-months mode** only when the user explicitly asks ("trend," "all months," "over time").

8.3 Governed LLM-SQL Fallback

For queries that fall outside the template library — comparative aggregations, cross-field analyses, custom balance range filters — the system falls back to LLM-generated SQL. This path opens a "can of worms" that the template approach cleanly avoids: an LLM with access to a database schema can, if not properly constrained, generate SQL that is destructive, exfiltrating, or structurally invalid.

The guardrail layer enforces the following rules on every LLM-generated SQL string before execution:

Table 3: SQL Guardrail Rules

Rule	Description
SELECT-only	Any statement not beginning with `SELECT` is rejected
No DML/DDL	Blocks `INSERT`, `UPDATE`, `DELETE`, `DROP`, `ALTER`, `TRUNCATE`, `CREATE`, `REPLACE`, `MERGE`
No comment chaining	`-`, `/**/`, and `;` are blocked to prevent injection
No `SELECT *`	Forces explicit column specification
Allowlist tables	Only permitted tables are queryable; `information_schema` and `mysql` system schemas blocked
Property binding	Query must include `:property_code` bound parameter
No hard-coded values	Prevents property code literal injection
Row limit	`LIMIT 100` enforced if absent

If any guardrail check fails, the system does not return an error to the user — it silently falls back to deterministic template behaviour. This graceful degradation is important: the user should never see a raw SQL validation error from a production interface.

8.4 SQL Provenance Tracking

Every SQL response includes machine-readable provenance metadata:

```
{
    "source_type": "sql",
    "property_code": "115R",
    "period_applied": "2025-05",
    "query_source": "template",          # or "
       llm_generated_validated"
    "sql_kind": "highest_balances",
    "row_count": 47
}
```

This allows downstream debugging, audit logging, and the synthesis node to correctly attribute data sources in citations.

9. Retrieval Design: Dense Vector with Hard Scope

The RAG retrieval path uses dense vector similarity (query embedding matched against stored chunk embeddings) with a mandatory metadata filter. The retrieval pipeline:

1. Embed the user query using `OllamaEmbeddings` (or Google as fallback).
2. Query Qdrant's `query_points` endpoint with the embedded query vector.
3. Apply a **hard filter**: `property_code == active_property_code`.
4. Return top-k matching chunks; deduplicate by `chunk_id`.
5. Package results with source citations including `source_url` and `chunk_id`.

9.1 Why Dense Over Alternatives

Table 4: Retrieval Strategy Comparison

Strategy	Strengths	Weaknesses for This Use Case
Dense vector (chosen)	Semantic matching; handles paraphrases; good on natural language property questions	Requires embedding infrastructure
Keyword/BM25	Simple, fast, no model needed	Breaks on synonyms; "parking" won't match "vehicle storage"
Hybrid lexical+dense	Best overall retrieval quality	Higher complexity to tune and operate; premature optimisation
Graph retrieval	Excellent for structured relationships	Overkill for website chunk RAG with no entity graph
Full-text SQL	No separate infrastructure	Weaker semantic relevance; harder to scale

The metadata filter is not just a performance optimisation — it is the primary tenancy mechanism for the RAG path. Without it, a query about "parking policy" could return chunks from any property in the collection. The filter makes the Qdrant collection multi-tenant without requiring separate collections per property.

10. Engineering Challenges and Resolutions

10.1 Idempotency Failure in Data Ingestion

Symptom: After running the ingest script a second time, the unit count doubled from 3,728 to 7,456 — every unit and charge record was duplicated.

Root Cause: The ingestion pipeline was using `INSERT` rather than `INSERT ... ON DUPLICATE KEY UPDATE` (or equivalent upsert semantics). Each execution appended new rows regardless of whether they already existed.

Resolution: The ingest script was made idempotent by keying on (property_code, snapshot_month, unit_number) as a composite natural key. Subsequent runs update existing records rather than creating duplicates. The admin endpoint supports a `mode=reload` parameter that clears and reinsertes for a full refresh when needed.

10.2 LangGraph State Key Collision

Symptom: The application crashed intermittently when transitioning between graph nodes. The error traced to a `route` key appearing in the shared LangGraph state object.

Root Cause: The `route` node was writing its routing decision into a state key named `route`. When a subsequent node read the state, LangGraph's internal routing logic read the same key, producing a collision between application state and framework-internal state.

Resolution: The routing decision was stored under a non-colliding key (`routing_decision`), and the application state schema was explicitly typed to prevent future collisions.

10.3 Mixed SQL Output in Synthesis

Symptom: The final synthesis response was sometimes partially correct — some rows came from the template SQL path, others from the LLM fallback, and they were interleaved in the output without clear provenance.

Root Cause: The SQL node was not consistently flushing its intermediate state before passing to synthesis. Both the template result and the LLM-generated result were present in the state simultaneously.

Resolution: The SQL node was refactored to produce a single canonical result with the `query_source` field explicitly set. The synthesis node was updated to read from a single well-defined state key.

10.4 Google API Quota and Model Deprecation

Symptom: The LLM inference layer began returning errors mid-session. The error messages were generic and did not clearly indicate quota exhaustion.

Root Cause: Google's free tier for the model version in use had either been deprecated or was hitting per-minute rate limits. The newer Gemini model family required a different model string.

Resolution: The model registry (`/models` endpoint) was introduced to abstract model selection. Users can switch the active LLM at runtime via the `model_id` field in the chat request. The system now supports Gemini, Grok (xAI), OpenAI, and Anthropic simultaneously, mitigating any single provider's availability risk.

11. System Deployment and Observability

11.1 Docker Compose Architecture

The full system runs in a single Docker Compose file, encompassing:

- **property_api** — FastAPI application container (Python 87.2% of the codebase)
- **property_mysql** — MySQL 8.x for structured data
- **qdrant** — Qdrant vector database on port 6333

Ollama runs on the host machine and is accessed from containers via `host.docker.internal:11434`, avoiding the need to containerise a GPU-dependent process.

11.2 Observability: Traces Endpoint

An admin traces endpoint (`/admin/traces`) records every query execution, including: routing decision, SQL mode used, row counts, RAG chunk counts, and synthesis latency. This enables post-hoc debugging without requiring a separate observability stack.

11.3 Testing Coverage

The test suite covers three critical path areas:

- `test_scope_and_period.py` — Verifies that every query is correctly scoped to the active property code and resolves date references correctly.
- `test_sql_citation_shape.py` — Verifies the shape and completeness of SQL provenance metadata in responses.
- `test_sql_guardrails.py` — Exercises the guardrail layer against known-bad SQL patterns (injection attempts, DML, missing property filter).

12. Orchestration Framework Selection: Full Evaluation

To provide complete context on the framework decision, the following table summarises the evaluation:

Table 5: Agent Orchestration Framework Evaluation

Framework	Strengths	Weaknesses	Selected
LangGraph	Explicit graph control; typed state; testable routing; provider-agnostic	Verbose; steeper learning curve	✓
CrewAI	Simple API; fast to prototype	Opaque scheduling; insufficient control for governed SQL	×
Anthropic SDK	Native Claude tool use; simple	Vendor lock-in; incompatible with zero-cost constraint	×
OpenAI SDK	Mature; good tooling	Same lock-in concern; cost risk	×
Custom	Full control	Significant engineering overhead	×

13. Future Work

The following improvements are already identified and architecturally viable within the current system:

13.1 Chunking Upgrade

Replace fixed-size character chunking with token-aware, structure-aware chunking that respects heading and paragraph boundaries. This will improve retrieval precision for the RAG path, particularly for website content with clear section structure. The metadata schema already supports this upgrade without changes to the Qdrant collection structure.

13.2 Conversational Memory Layer

The current system treats each query as stateless. Adding a short-term memory layer (e.g., a sliding window of recent exchanges) would enable follow-up questions: "What about unit 205 specifically?" after "Show me all vacant units." LangGraph's state model makes this architecturally straightforward — memory would live in the shared state object and be passed through the graph on each turn.

13.3 Automated Data Refresh Agents

The current ingest pipeline is manual. An agent that monitors for new Excel exports, detects schema changes, and runs the idempotent ingest automatically would close the loop on the operational side.

13.4 Nearby Market Intelligence

A separate agent that crawls public market data (competing properties, neighbourhood developments, transit changes) could extend the RAG corpus beyond each property's own website, enabling questions like "How does our market rent compare to nearby properties?"

13.5 Hybrid Retrieval

Augmenting the current dense retrieval with a BM25 lexical component (hybrid retrieval) would improve recall on exact-match queries where a user includes a specific unit number or address in a website-content question.

14. Summary of Key Design Decisions

Table 6: Key Design Decisions and Rationale

Decision	Alternatives	Rationale	Outcome
Deterministic SQL templates as primary path	LLM-generated SQL	Production reliability; precision over recall	Stable, auditable, fast
Governed LLM-SQL fallback	No SQL fallback	Flexibility for non-template queries	Extended coverage
Separate MySQL + Qdrant	Unified pgvector or single DB	Different access patterns; clean separation of concerns	Clean architecture
Qdrant for vector store	Pinecone, pgvector, Weaviate	Self-hosted, strong metadata filtering, low ops cost	Free, reliable
Local Ollama embeddings	Cloud embedding APIs	Quota-free, consistent, zero marginal cost	Eliminated API failures
LangGraph orchestration	CrewAI, direct SDK	Explicit control; testable; provider-agnostic	Governed, debuggable
SQL-first HYBRID routing	Parallel SQL + RAG	Grounding + TTFT optimisation	Better UX, more accurate synthesis
Fixed-size chunking	Semantic chunking	Simplicity for first pass; explicit upgrade path	Fast, deterministic

Conclusion

This project explores how modern AI systems can combine deterministic data retrieval, semantic search, and agentic orchestration while remaining reliable enough for production use. The architecture intentionally prioritizes correctness and traceability before sophistication.

A central design principle emerged throughout development: structured data should be retrieved deterministically whenever possible, while language models should act as reasoning and synthesis layers rather than sources of truth. This approach reduces hallucinations, improves auditability, and provides a clearer path toward production deployment.

The resulting system combines SQL-based retrieval, vector search, runtime model switching, local embeddings, and graph-based orchestration into a single extensible architecture. While there are many opportunities for future improvements—including hybrid retrieval, conversational memory, and automated knowledge ingestion—the current design provides a strong foundation for building trustworthy AI assistants in real-world environments.

Disclaimer: Final draft written by a myself and polished with AI assistance.